

Elementy T-SQL: widoki i procedury

Ćw. Nr 13

T-SQL

Transact SQL (T-SQL) - rozszerzenie standardowego języka SQL używane w systemie MS SQL Server.

Umożliwia nie tylko definiowanie i manipulowanie danymi ale również tworzenie złożonej logiki programistycznej w środowisku bazodanowym.

Funkcje T-SQL:

- Deklarowanie i używanie zmiennych
- Możliwość tworzenia funkcji, procedur, triggerów
- Obsługa transakcji
- Instrukcje sterujące jak IF ...ELSE, WHILE, TRY ...CATCH
- Dynamiczne tworzenie i wykonywanie zapytań
- Korzystanie z kursorów

SQL a T-SQL

Standardowy SQL pozwala głównie na:

- pobieranie danych (SELECT)
- modyfikację danych (INSERT, UPDATE, DELETE)
- definiowanie struktur (CREATE, ALTER)

T-SQL dodaje do tego **elementy programistyczne**, dzięki którym można tworzyć bardziej złożoną logikę po stronie serwera bazy danych.

Najważniejsze cechy T-SQL

❑ ZMIENNE

```
DECLARE @wiek INT = 20;
```

❑ INSTRUKCJE WARUNKOWE

```
IF @wiek >= 18
```

```
    PRINT 'Pełnoletni';
```

❑ PĘTLE

```
WHILE @i <= 10
```

❑ OBSŁUGA TRANSAKCJI

```
BEGIN TRAN;
```

```
COMMIT;
```

```
ROLLBACK;
```

❑ PROCEDURY SKŁADOWANE

```
CREATE PROCEDURE  
dbo.ListaStudentow
```

```
AS
```

```
    SELECT * FROM Studenci;
```

❑ FUNKCJE UŻYTKOWNIKA

```
CREATE FUNCTION  
dbo.Podatek(@kwota FLOAT)
```

```
RETURNS FLOAT
```

❑ OBSŁUGA BŁĘDÓW

```
TRY...CATCH
```

Zastosowanie T-SQL

Gdzie używa się T-SQL?

- Microsoft SQL Server
- Azure SQL Database
- SQL Server Management Studio (SSMS)

Do czego służy T-SQL?

- automatyzacja operacji na bazie danych
- tworzenie procedur i funkcji
- kontrola transakcji
- walidacja danych
- logika biznesowa po stronie serwera

Zmienne

- ❑ Zmienne w języku SQL, podobnie jak w innych językach programowania, służą do przechowywania wartości w pamięci. W języku SQL, zmienne są często używane w procedurach, aby móc modyfikować wartości i wykorzystać je w kolejnych instrukcjach.
- ❑ Zmienne w języku SQL są definiowane za pomocą instrukcji DECLARE, która określa nazwę zmiennej i jej typ danych.

```
declare @i int, @j int;  
declare @first varchar(50);  
declare @last varchar(50);
```

- ❑ Wartości zmiennym przypisywane są za pomocą instrukcji SET

```
set @i = 10;  
set @j = @i + 5;
```

Podstawianie wartości pod zmienną

```
declare @first varchar(50);  
declare @last varchar(50);  
set @first = (select firstname from employees where employeeid = 1);  
print @first  
select @last = lastname from employees where employeeid = 1;  
print @last;  
select @first = firstname, @last = lastname from employees where  
employeeid = 2;  
print @first + ' ' + @last;
```

Podstawianie wartości pod zmienną c.d.

- ❑ Co jeśli polecenie select zwróci kilka wierszy?

```
set @first = (select firstname from employees where employeeid > 1);
```

```
print @first;
```

- zakończy się błędem

Subquery returned more than 1 value. This is not permitted when the subquery follows =, !=, <, <=, >, >= or when the subquery is used as an expression.

```
Select @last = lastname from employees where employeeid > 1);
```

```
print @last;
```

- zwróci wartość z ostatniego z wierszy przetwarzanych przez polecenie
select * from employees where employeeid > 1

Widoki

Widoki (perspektywy) - obiekty w bazie danych - wirtualne tabele

Przechowują zapytania SQL

Mogą łączyć dane z wielu tabel

Mogą dokonywać agregacji, filtrowania czy sortowania

Nie przechowują danych - zawierają zapytanie wykonywane za każdym razem gdy widok jest wykorzystywany

Widoki c.d.

Zadania realizowane przez widoki:

- Definiowanie takiej struktury danych, która umożliwi pojedynczym użytkownikom i całym klasom użytkowników wygodne korzystanie z systemu.
- Ograniczenie dostępu do danych, tak aby użytkownik mógł przeglądać (ew. modyfikować) tylko te informacje, które będą tego wymagały.

Widoki c.d.

Zalety stosowania widoków:

- ❑ Ułatwiają zarządzanie i prezentację danych.
- ❑ Zwiększają bezpieczeństwo (można ograniczyć dostęp do części danych).
- ❑ Mogą uprościć skomplikowane zapytania.

Widoki w SQL pozwalają na uporządkowanie i uproszczenie dostępu do danych, ułatwiając użytkownikom baz danych korzystanie z informacji w sposób bardziej strukturalny i bezpieczny.

Widoki c.d.

Jakich instrukcji można używać w widokach:

- **SELECT, JOIN, WHERE, GROUP BY**
- funkcje agregujące (**COUNT, SUM**)
- korzystać z innych widoków

Czego nie można używać w standardowych widokach:

- **ORDER BY** (bez TOP)
- zmiennych
- instrukcji sterujących (**IF, WHILE**)
- transakcji

Tworzenie widoku

```
CREATE VIEW NazwaWidoku AS  
SELECT Kolumna1, Kolumna2  
FROM Tabela  
WHERE Warunki;
```

```
CREATE VIEW <nazwa widoku> AS <definicja widoku>
```

Tworzenie widoku c.d.

Przykład:

```
create view vw_emp_names  
as  
select firstname + ' ' + lastname as  
name  
from employees;
```

Użycie widoku:

```
select * from vw_emp_names;
```

	name
1	Nancy Davolio
2	Andrew Fuller
3	Janet Leverling
4	Margaret Peacock
5	Steven Buchanan
6	Michael Suyama
7	Robert King
8	Laura Callahan
9	Anne Dodsworth

Widoki

- ❑ Modyfikacja widoku - **alter view** :

```
create or alter view vw_emp_names  
as  
select employeeid, firstname + ' ' + lastname as name  
from employees;
```

- ❑ Usuwanie widoku - **drop view** :

```
drop view vw_emp_names;
```

Procedury składowane

- ❑ **Procedury składowane** (ang. stored procedure) - to przygotowany kod SQL, który może być wielokrotnie używany. Może zawierać wiele instrukcji (SELECT, INSERT, UPDATE).
- ❑ Format tworzenia procedury:

```
CREATE PROCEDURE NazwaProcedury  
AS  
  DeklaracjeOpcjonalnychParametrów  
BEGIN  
  InstrukcjeSQL  
END
```

- ❑ Wywołanie procedury składowanej:

```
EXEC NazwaProcedury
```


Procedury składowane - zalety

- ❑ UPROSZCZENIE ZŁOŻONYCH OPERACJI - dzięki hermetyzacji procesów w jednej, łatwej w użyciu jednostce.
- ❑ POWSTANIE BARDZO ELASTYCZNEGO I UŻYTECZNEGO KODU.
- ❑ ZAPEWNIENIE SPÓJNOŚCI DANYCH - nie jest potrzebne wielokrotne wykonywanie tej samej sekwencji działań.
- ❑ ZMNIEJSZENIE RYZYKA POWSTAWANIA BŁĘDÓW (im więcej kroków do wykonania tym większe prawdopodobieństwo pojawienia się błędu).

Procedury składowane - zalety

- ❑ UŁATWIENIE DOKONYWANIA ZMIAN. Jeżeli tabele, nazwy kolumn, logika biznesowa ulegną zmianie, modyfikacji wystarczy dokonać tylko w kodzie procedury składowanej.
- ❑ ZWIĘKSZENIE BEZPIECZEŃSTWA - ograniczenie dostępu do szczegółowych danych dzięki procedurom zmniejsza prawdopodobieństwo uszkodzenia danych.
- ❑ OGRANICZENIE PRACY POTRZEBNEJ PRZY PRZETWARZANIU POLECENIA.

Parametry w procedurach składowanych

Parametry w procedurach składowanych służą do:

- ❑ przekazywania danych **do** procedury,
- ❑ zwracania danych **z** procedury,
- ❑ sterowania logiką wykonania procedury.

Ilość i typ danych, które należy podać przy wywołaniu procedury składowanej określamy w trakcie tworzenia procedury (używając polecenia CREATE PROCEDURE)

Rodzaje parametrów:

- ❑ Wejściowe (INPUT)
- ❑ Wyjściowe (OUTPUT)
- ❑ Przejściowe

Parametry wejściowe (INPUT)

- ❑ domyślny typ parametrów
- ❑ przekazują dane do procedury
- ❑ nie wymagają dodatkowych słów kluczowych

Przykład:

```
CREATE OR ALTER PROC
p_customer_orders
    @CustomerID NCHAR(5)
AS
BEGIN
    SELECT OrderID, OrderDate
    FROM Orders
    WHERE CustomerID =
@CustomerID;
END;
```

Wywołanie:

```
EXEC p_customer_orders 'ALFKI';
```

Parametry wyjściowe (OUTPUT)

- ❑ służą do zwracania pojedynczych wartości
- ❑ muszą być oznaczone słowem kluczowym OUTPUT
- ❑ wymagają zmiennej przy wywołaniu procedury

Przykład - liczba zamówień klienta;

```
CREATE PROC p_order_count
    @CustomerID NCHAR(5),
    @OrderCount INT OUTPUT
AS
BEGIN
    SELECT @OrderCount = COUNT(*)
    FROM Orders
    WHERE CustomerID = @CustomerID;
END;
Wywołanie:
DECLARE @Liczba INT;
EXEC p_order_count 'ALFKI', @Liczba
OUTPUT;
SELECT @Liczba AS LiczbaZamowien;
```

Procedury składowane c.d.

- **alter** proc - modyfikacja procedury
- **drop** proc - usunięcie procedury

Przykład procedury - pobieranie zamówień danego klienta:

```
CREATE OR ALTER PROC p_customer_orders
    @CustomerID NCHAR(5)
AS
BEGIN
    SELECT OrderID, OrderDate, ShipCity, Freight
    FROM Orders
    WHERE CustomerID = @CustomerID;
END;

EXEC p_customer_orders 'CACTU';
```

Procedury składowane - przykład:

- ❑ Tworzymy procedurę, która sprawdzi czy klient istnieje. Jeśli tak - wyświetli jego dane, jeśli nie - zwróci komunikat.

```
CREATE OR ALTER PROC p_klienci_w_bazie
    @CustomerID NCHAR(5)
AS
BEGIN
    IF EXISTS (SELECT 1 FROM Customers WHERE CustomerID
= @CustomerID)
        SELECT * FROM Customers WHERE CustomerID =
@CustomerID;
    ELSE
        PRINT 'Klient nie istnieje';
END;
```

Wywołanie:

```
EXEC p_klienci_w_bazie 'ALFKI'
```

Procedury składowane - przykład:

- ❑ Utwórz procedurę składowaną, która dodaje nowe zamówienie do tabeli Orders. Procedura powinna przyjmować następujące parametry:

@CustomerID, @EmployeeID, @OrderDate, @ShipVia, @Freight, @ShipCity

```
CREATE OR ALTER PROC p_add_order
    @CustomerID NCHAR(5),
    @EmployeeID INT,
    @OrderDate DATE,
    @ShipVia INT,
    @Freight MONEY,
    @ShipCity NVARCHAR(15)
AS
BEGIN
    INSERT INTO Orders (CustomerID, EmployeeID, OrderDate, Freight, ShipCity)
        VALUES (@CustomerID, @EmployeeID, @OrderDate, @Freight, @ShipCity);
    PRINT 'Zamówienie zostało dodane';
END;
```


Procedury składowane - przykład c.d.

❑ Wywołanie procedury:

```
EXEC p_add_order_2
  @CustomerID = 'ALFKI',
  @EmployeeID = 4,
  @OrderDate = '2026-01-13',
  @ShipVia = 2,
  @ShipCity = 'London',
  @Freight = 50.50;
```

❑ Sprawdzenie działania:

```
SELECT TOP 1 *
FROM Orders
ORDER BY OrderID DESC;
```

Results Messages											
	OrderID	CustomerID	EmployeeID	OrderDate	RequiredDate	ShippedDate	ShipVia	Freight	ShipName	ShipAddress	ShipCity
1	12080	ALFKI	4	2026-01-13 00:00:00.000	NULL	NULL	2	50,50	NULL	NULL	London

Widoki - ćwiczenia I

1. Stwórz widok, który zwróci listę zamówień z nazwami klientów, datami zamówień i datą wysyłki.
2. Utwórz widok łączący pracowników i zamówienia przez nich obsługiwane.
3. Stwórz widok z podsumowaniem sprzedaży według kategorii (widok agreguje dane sprzedaży według kategorii produktów).

Widoki - ćwiczenia II

1. Utwórz widok o nazwie `vw_products_ok`, który będzie zawierał podstawowe informacje o produktach dostępnych w sprzedaży. Widok powinien zawierać tylko te produkty, które nie zostały wycofane ze sprzedaży (czyli pole `Discontinued` ma wartość 0).
2. Utwórz widok, który będzie wyświetlał tylko te produkty, które są na magazynie (`unitsinstock > 0`). Następnie napisz polecenie `select` z użyciem tego widoku, która wyświetli liczbę takich produktów.

Procedury składowane - ćwiczenia I

1. Napisz procedurę p_AddNewOrder, która dodaje nowe zamówienie do tabeli Orders. Procedura powinna przyjmować parametry:
 - @CustomerID (typ nchar(5)),
 - @EmployeeID (typ int),
 - @ShipVia (typ int),
 - @OrderDate
 - Data zamówienia ma być bieżącą datą (GETDATE()), a termin realizacji (RequiredDate) - 7 dni później.
 - Wykonaj procedurę używając przykładowych danych.
 - Sprawdź poprawność jej działania.

Procedury składowane - ćwiczenia II

1. Utwórz procedurę p_UnitsInStockUpdate, która przyjmuje dwa parametry:

@ProductID (int)

@Nowallosc (smallint)

i aktualizuje kolumnę UnitsInStock w tabeli Products.

Wykonaj procedurę dla ProductID=1, wprowadzając nową ilość

2. Utwórz procedurę o nazwie p_ProductSales , która przyjmuje parametr

@ProductID (int)

zwraca nazwę produktu i sumaryczną wartość sprzedaży na podstawie tabeli Order Details.

Procedury składowane - ćwiczenia III

1. Napisz procedurę p_AddCustomer, która dodaje nowy rekord do tabeli Customers. Procedura powinna przyjmować przynajmniej cztery parametry:

@CustomerID (nchar(5))

@CompanyName (nvarchar(40))

@ContactName (nvarchar(30))

@City (nvarchar(15))

Dodaj też walidację: jeśli klient nie istnieje jeszcze w bazie - dodaj go, jeśli już istnieje - nie wykonuj dodania, wyświetl komunikat „Klient o podanym ID już istnieje”.

Procedury składowane - ćwiczenia IV

1. Utwórz procedurę, która wyświetla wszystkie produkty z danej kategorii, wraz z informacją o cenie jednostkowej i stanach magazynowych tych produktów. Przyjmij nazwę kategorii jako parametr.

Sprawdź działanie procedury dla kategorii Confections.

2. Utwórz procedurę, która pokazuje statystyki sprzedaży pracowników w danym roku. Jako parametr procedura przyjmuje EmployeeID oraz rok i zwraca liczbę obsłużonych zamówień oraz ich łączną wartość.

Sprawdź działanie procedury dla wybranego pracownika.